Clasificación del riesgo de caída en adultos mayores mediante análisis de marcha y aprendizaje automático: avance de la sub-etapa 1.3, implementación y entrenamiento de modelos

***Resumen*: Este documento presenta el avance de la sub-etapa 1.3 del proyecto, correspondiente a las semanas 6 a 9. Con el dataset limpio de la sub-etapa 1.2 (72 personas de la base LTMM) se implementaron y entrenaron los tres modelos comprometidos: Random Forest y XGBoost, que usan 14 variables de marcha, y una red neuronal LSTM con dos capas convolucionales previas, que usa directamente las señales del sensor. Los modelos se evaluaron con validación cruzada estratificada y repetida, separando siempre por persona, y se compararon con tres referencias simples. Random Forest obtuvo un AUC de 0,68 ± 0,14 y XGBoost de 0,64 ± 0,15, mientras que la LSTM quedó cerca del azar (0,55 ± 0,15) por un sobreajuste marcado. El mejor resultado lo obtuvo una regresión logística con una sola variable, la frecuencia dominante de la marcha (AUC 0,75 ± 0,13), que además superó a la prueba clínica TUG (0,66 ± 0,14). Con 72 personas, el desempeño empeora a medida que aumenta la complejidad del modelo. Los tres modelos quedaron guardados y documentados en notebooks reproducibles.**

***Palabras clave*: riesgo de caída, análisis de marcha, Random Forest, XGBoost, LSTM, validación cruzada, sobreajuste.**

I. Introducción

La sub-etapa 1.3 tiene como objetivo implementar y entrenar los tres modelos de aprendizaje automático definidos en el proyecto: Random Forest [3], XGBoost [4] y LSTM [5]. El producto comprometido es un notebook reproducible con los tres modelos entrenados, versionado en un repositorio de GitHub.

El trabajo parte del dataset limpio de la sub-etapa 1.2, obtenido de las caminatas de laboratorio de la base Long Term Movement Monitoring (LTMM) [1], [2]. Tras el control de calidad quedaron 72 personas: 38 que no caen y 34 con dos o más caídas en el año previo. Para cada persona se dispone de 14 variables de marcha no redundantes y de sus señales limpias de seis canales. La base CASIA no se usó en esta sub-etapa porque no tiene etiquetas de caída; se reserva para validar la estimación de pose en la Etapa 2.

El documento se organiza así. La Sección II describe la metodología. La Sección III presenta los resultados y la Sección IV los discute. La Sección V resume las limitaciones y los próximos pasos, y la Sección VI, los productos entregados.

II. Metodología

A. Validación

Con solo 72 personas, una única división entre entrenamiento y prueba daría un resultado que depende mucho de qué personas quedan en cada grupo. Por eso se usó validación cruzada estratificada de 5 partes: los datos se dividen en 5 grupos con la misma proporción de personas que caen, se entrena con 4 y se prueba con el restante, rotando hasta usar cada grupo una vez. Para Random Forest, XGBoost y las referencias, este proceso se repitió 10 veces con un orden distinto, lo que da 50 pruebas por modelo. La LSTM, mucho más lenta de entrenar, se evaluó con 3 repeticiones (15 pruebas), usando exactamente las

mismas particiones que las primeras 15 de los otros modelos, lo que permite compararlos en igualdad de condiciones.

Las personas de prueba nunca se usan durante el entrenamiento, como recomiendan Saeb et al. [8]. En la tabla de variables cada persona aparece una sola vez, por lo que la separación por persona es automática; en la LSTM, todas las ventanas de una persona quedan siempre del mismo lado. Los pasos que aprenden algo de los datos, como completar valores faltantes o normalizar las señales, se calcularon solo con los datos de entrenamiento de cada partición. Se reportan exactitud, sensibilidad, especificidad, F1-score y AUC, como promedio ± desviación estándar de las pruebas.

B. Puntos de comparación

Un modelo solo es útil si supera a alternativas más simples. Por eso se incluyeron tres referencias: un clasificador sin modelo, que responde siempre la clase más frecuente (AUC 0,5); una regresión logística con una sola variable, la frecuencia dominante, que fue la única con diferencia significativa en el análisis exploratorio; y una regresión logística con el resultado de la prueba Timed Up and Go (TUG) de la planilla clínica, disponible en 68 de las 72 personas.

C. Random Forest y XGBoost

Ambos modelos usan árboles de decisión y reciben las 14 variables de marcha. Random Forest entrena muchos árboles independientes y decide por votación [3]; XGBoost construye los árboles uno tras otro, de modo que cada uno corrige los errores de los anteriores [4]. Se eligieron parámetros conservadores, pensados para pocos datos, que se resumen en la Tabla I. Su ajuste fino queda para la sub-etapa 1.4.

TABLA I. Parámetros de Random Forest y XGBoost

Modelo	Parámetro	Motivo
Random Forest	500 árboles	Resultado estable entre ejecuciones
Random Forest	Mínimo 2 personas por hoja	Evita que los árboles memoricen casos aislados
Random Forest	Pesos de clase equilibrados	Mismo peso a quienes caen y no caen
XGBoost	300 árboles, tasa de aprendizaje 0,05	Muchos árboles pequeños que aprenden de a poco
XGBoost	Profundidad máxima 3	Árboles simples, adecuados para pocos datos
XGBoost	80 % de personas y variables por árbol	Reduce el sobreajuste

D. Red CNN + LSTM

A diferencia de los otros modelos, la LSTM recibe directamente las señales del sensor. Para aumentar el número de ejemplos, cada caminata se cortó en ventanas de 5 s, a 50 Hz y con los seis canales, traslapadas

a la mitad. Así se obtuvieron 1150 ventanas (entre 8 y 44 por persona, con una mediana de 15). Cada ventana lleva la etiqueta de su persona, y la probabilidad de caer de una persona se calcula como el promedio de sus ventanas.

En una prueba previa con datos simulados, una LSTM sola sobre las señales crudas no lograba aprender. Por eso se antepusieron dos capas convolucionales, que detectan patrones cortos de la señal y la resumen antes de que la lea la LSTM, siguiendo la arquitectura que usaron Kiprijanovska et al. [6] y Alqurashi et al. [7]. La red completa tiene dos convoluciones de 16 y 32 filtros, una LSTM de 32 unidades y dos capas de salida, con un total de 12 081 parámetros. Se entrenó con el optimizador Adam, una tasa de aprendizaje de 0,001 y dropout de 0,3. Un 20 % de las personas de entrenamiento se apartó para decidir cuándo detener el entrenamiento: la red entrena al menos 40 épocas y se detiene si la pérdida de validación no mejora en 20, conservando la mejor versión.

E. Herramientas y reproducibilidad

Los modelos se implementaron en Python con scikit-learn [9], XGBoost [4] y PyTorch [10]. Se eligió PyTorch en lugar de TensorFlow porque este último todavía no tiene versión para Python 3.14. Todos los procesos usan la semilla aleatoria 42 y cada notebook se ejecuta completo de principio a fin. Al terminar, cada modelo se entrenó con todas las personas y se guardó como checkpoint junto con un archivo que registra sus variables de entrada, parámetros, métricas de validación y versiones de las librerías.

III. Resultados

A. Random Forest y XGBoost

La Tabla II muestra los resultados de las 50 pruebas. Todos los modelos superaron a la referencia sin modelo. Random Forest obtuvo un AUC de 0,68 y XGBoost de 0,64, con sensibilidades de 0,60 y 0,55. Sin embargo, la regresión con solo la frecuencia dominante superó a ambos, con un AUC de 0,75 y la mayor exactitud (0,70). La prueba TUG fue la referencia más débil, con un AUC de 0,66 y una sensibilidad de solo 0,40.

TABLA II. Resultados de Random Forest, XGBoost y las referencias (50 pruebas)

Modelo	Exactitud	Sensibilidad	Especificidad	F1	AUC
Sin modelo	0,53 ± 0,03	0,00 ± 0,00	1,00 ± 0,00	0,00 ± 0,00	0,50 ± 0,00
Solo frecuencia dominante	**0,70 ± 0,12**	0,60 ± 0,19	**0,80 ± 0,13**	**0,65 ± 0,15**	**0,75 ± 0,13**
Solo TUG	0,56 ± 0,13	0,40 ± 0,19	0,71 ± 0,20	0,45 ± 0,16	0,66 ± 0,14
Random Forest	0,61 ± 0,11	0,60 ± 0,17	0,61 ± 0,18	0,59 ± 0,12	0,68 ± 0,14
XGBoost	0,59 ± 0,13	0,55 ± 0,19	0,64 ± 0,19	0,55 ± 0,15	0,64 ± 0,15

Las desviaciones estándar son grandes (entre 0,13 y 0,15 en el AUC), lo que muestra que el resultado cambia bastante según qué personas quedan en la prueba. Las diferencias entre los modelos son menores que esa variación, por lo que deben confirmarse con una comparación formal en la sub-etapa 1.4.

B. LSTM

La LSTM obtuvo un AUC de 0,55 ± 0,15, apenas sobre el azar. Las curvas de aprendizaje (Fig. 1) explican este resultado: la pérdida de entrenamiento baja casi hasta cero, mientras que la de validación aumenta desde las primeras épocas. En las tres particiones graficadas, la mejor versión de la red fue la de las épocas 2 a 4, y en las 15 particiones el entrenamiento se detuvo en el mínimo de 40 épocas. Es decir, la red aprendió a reconocer a las personas de entrenamiento, pero lo aprendido no le sirvió para clasificar a personas nuevas.

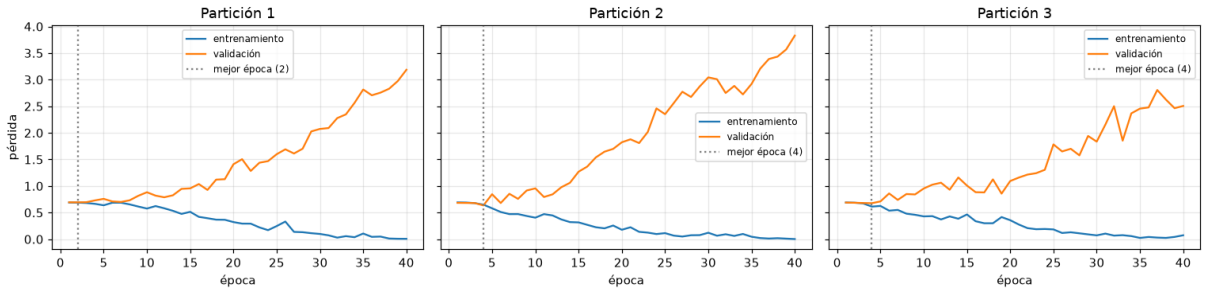


Fig. 1. Curvas de aprendizaje de la LSTM en las tres primeras particiones. La pérdida de entrenamiento baja mientras la de validación sube desde las primeras épocas, lo que indica sobreajuste. La línea punteada marca la mejor época.

C. Comparación de los tres modelos

La Tabla III y la Fig. 2 comparan todos los modelos en las mismas 15 particiones. El orden se mantiene: la frecuencia dominante sola obtuvo el mayor AUC (0,75), seguida de Random Forest (0,68), XGBoost (0,67), el TUG (0,66) y la LSTM (0,55). La LSTM tuvo además la sensibilidad más variable, entre 0 y 1 según la partición.

TABLA III. Comparación de los tres modelos en las mismas 15 pruebas

Modelo	Exactitud	Sensibilidad	Especificidad	F1	AUC
Sin modelo	0,53 ± 0,03	0,00 ± 0,00	1,00 ± 0,00	0,00 ± 0,00	0,50 ± 0,00
Solo frecuencia dominante	**0,71 ± 0,13**	0,60 ± 0,23	**0,80 ± 0,10**	**0,64 ± 0,19**	**0,75 ± 0,15**
Solo TUG	0,56 ± 0,12	0,39 ± 0,17	0,70 ± 0,22	0,45 ± 0,15	0,66 ± 0,13
Random Forest	0,64 ± 0,11	0,64 ± 0,17	0,65 ± 0,16	0,62 ± 0,12	0,68 ± 0,14
XGBoost	0,60 ± 0,10	0,58 ± 0,23	0,62 ± 0,19	0,56 ± 0,17	0,67 ± 0,13
LSTM	0,53 ± 0,07	0,58 ± 0,36	0,48 ± 0,38	0,47 ± 0,24	0,55 ± 0,15

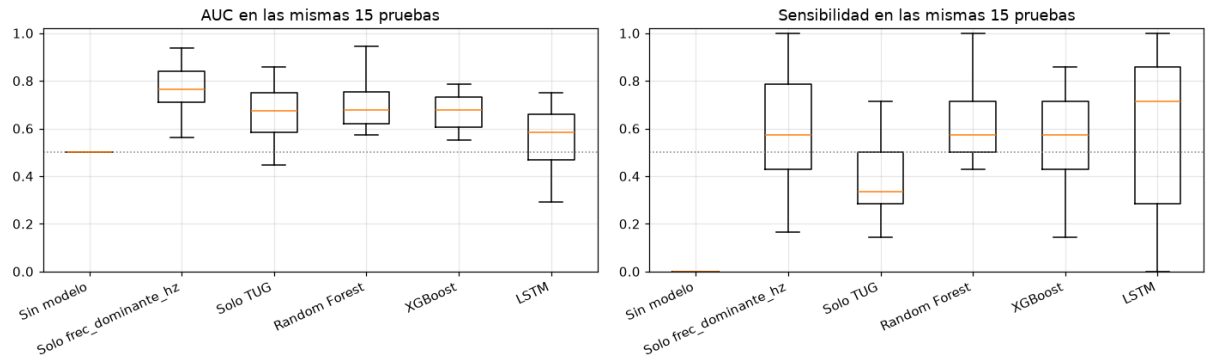


Fig. 2. AUC y sensibilidad de cada modelo en las mismas 15 pruebas de la validación cruzada. La línea punteada marca el nivel del azar (0,5).

D. Variables más importantes

En los modelos finales, entrenados con todas las personas, la variable más importante fue la frecuencia dominante, tanto en Random Forest como en XGBoost (Fig. 3). En Random Forest le siguieron la cadencia y la amplitud dominante. Es decir, los modelos complejos llegaron a la misma conclusión que el análisis exploratorio: el ritmo de la marcha es lo que más distingue a quienes caen.

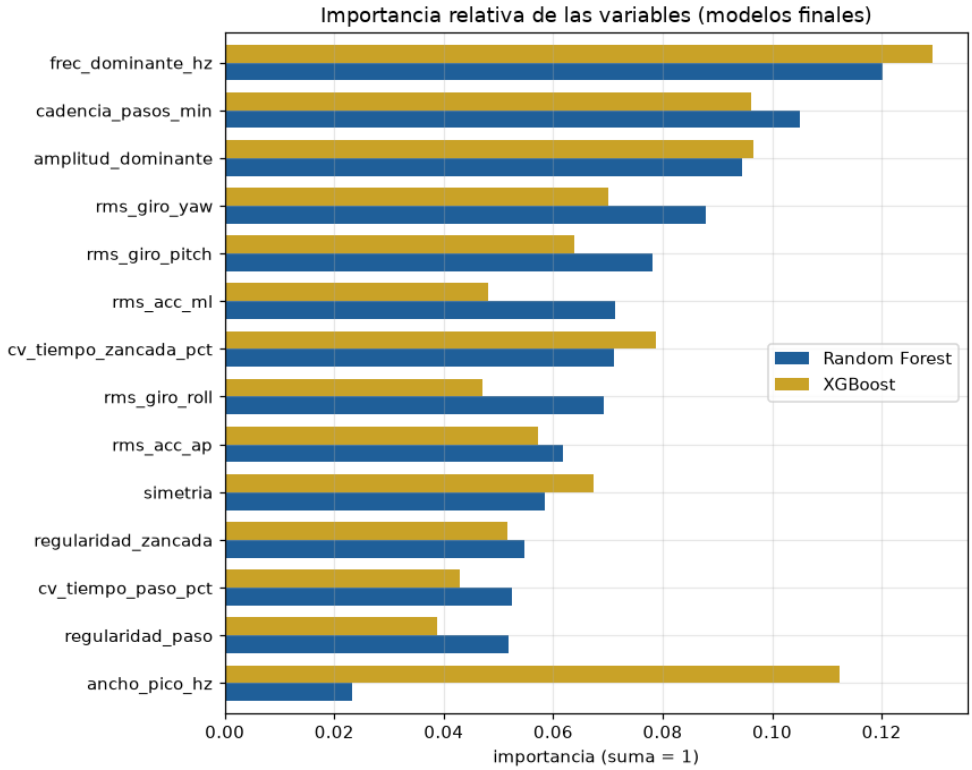


Fig. 3. Importancia relativa de las variables en los modelos finales de Random Forest y XGBoost (cada modelo suma 1).

IV. Discusión

El resultado principal es que, en esta muestra, el desempeño empeora a medida que aumenta la complejidad del modelo: una sola variable supera a Random Forest y XGBoost, y estos superan a la LSTM. La explicación más probable es el sobreajuste. Con 72 personas, los modelos complejos aprenden también del ruido de las variables que casi no distinguen a los grupos, y la LSTM, con más de 12 000 parámetros, llega a memorizar a cada persona. Aunque las ventanas dan 1150 ejemplos, provienen de solo 72 personas, que es la cantidad real de casos independientes. Esto concuerda con la literatura revisada: el metaanálisis de Mou et al. [11] advierte que los estudios con menos de 100 participantes tienden a reportar resultados demasiado optimistas, y Alqurashi et al. [7] obtuvieron buenos resultados con redes LSTM usando 163 personas.

La marcha distinguió a quienes caen mejor que la prueba TUG (AUC 0,75 frente a 0,66), lo que concuerda con la capacidad limitada de esta prueba descrita por Bargiotas et al. [13]. En cuanto a la literatura, Altunkaya [12] reportó un 82,2 % de exactitud con las mismas caminatas de LTMM. La diferencia con este trabajo (hasta 0,71) probablemente se debe al esquema de validación y a la forma de seleccionar las variables, por lo que la comparación es solo orientativa. También debe considerarse que la frecuencia dominante se eligió a partir del análisis exploratorio hecho con todas las personas, por lo que su desempeño puede estar algo sobreestimado.

Con un AUC de 0,68 a 0,75, los modelos distinguen a los grupos mejor que el azar y que el TUG, pero todavía no lo suficiente para usarse como herramienta clínica. Además, como las etiquetas de LTMM son retrospectivas, los modelos reconocen a quienes ya cayeron, pero no predicen caídas futuras. Un resultado favorable para el proyecto es que la variable más informativa, el ritmo de la marcha, se puede medir con una cámara, como ya se comprobó con las siluetas de CASIA en la sub-etapa 1.2. Esto respalda la idea de llevar el modelo a un prototipo con cámara en la Etapa 2.

V. Limitaciones y próximos pasos

La principal limitación es la cantidad de datos por persona: cada una aporta una sola caminata de laboratorio de alrededor de un minuto. A esto se suman las etiquetas retrospectivas y la gran variación de los resultados entre particiones. Para la sub-etapa 1.4 se proponen los siguientes pasos:

- **Comparación formal.** Comparar los modelos partición por partición, para saber si las diferencias observadas son reales o se deben al azar, y seleccionar el modelo final.
- **Menos variables.** Probar modelos con las variables más importantes, eligiéndolas dentro de la validación cruzada para no repetir el sesgo de elegirlas mirando todos los datos.
- **Registros de tres días.** La base LTMM incluye, además de las caminatas de laboratorio, registros de tres días de vida diaria de las mismas personas [1], [2]. Weiss et al. encontraron en ellos las diferencias más claras entre grupos, y aportan mucha más marcha por persona, que es justamente lo que le faltó a la LSTM.
- **LSTM más simple.** Probar una red más pequeña y más regularizada, aunque con esta cantidad de personas es poco probable que supere a los modelos simples.

VI. Productos

La Tabla IV lista los productos de esta sub-etapa, disponibles en el repositorio del proyecto.

TABLA IV. Productos de la sub-etapa 1.3

Archivo	Contenido
notebooks/03_Modelos_RF_XGBoost.ipynb	Referencias, Random Forest y XGBoost: entrenamiento, validación y comparación
notebooks/04_Modelo_LSTM.ipynb	Ventanas de señal, red CNN + LSTM, validación y comparación de los tres modelos
modelos/random_forest.joblib, xgboost.joblib	Modelos finales de árboles (checkpoints)
modelos/lstm.pt	Modelo final de la red (checkpoint)
modelos/info_modelos.json, info_lstm.json	Variables de entrada, parámetros, normalización, métricas y versiones
resultados/*.csv	Métricas por partición y resumen de los tres modelos
figuras/modelos_*.png, lstm_*.png	Gráficos de resultados

Referencias

[1] A. Weiss, M. Brozgol, M. Dorfman, T. Herman, S. Shema, N. Giladi, and J. M. Hausdorff, "Does the evaluation of gait quality during daily life provide insight into fall risk? A novel approach using 3-day accelerometer recordings," Neurorehabil. Neural Repair, vol. 27, no. 8, pp. 742–752, 2013, doi: 10.1177/1545968313491004.

[2] J. M. Hausdorff, "Long Term Movement Monitoring Database," version 1.0.0, PhysioNet, 2016, doi: 10.13026/C2S59C.

[3] L. Breiman, "Random forests," Mach. Learn., vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[4] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining, San Francisco, CA, USA, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.

[6] I. Kiprijanovska, H. Gjoreski, and M. Gams, "Detection of gait abnormalities for fall risk assessment using wrist-worn inertial sensors and deep learning," Sensors, vol. 20, no. 18, Art. no. 5373, Sep. 2020, doi: 10.3390/s20185373.

[7] A. Alqurashi, A. Alharthi, M. M. Alammar, N. Aldosari, and A. Al Ayidh, "Classification of fallers and non-fallers in older adults using electrical IMU signal for gait analysis and explainable deep learning," Sci. Rep., vol. 16, no. 1, Art. no. 14353, Mar. 2026, doi: 10.1038/s41598-026-44368-w.

[8] S. Saeb, L. Lonini, A. Jayaraman, D. C. Mohr, and K. P. Kording, "The need to approximate the use-case in clinical machine learning," GigaScience, vol. 6, no. 5, Art. no. gix019, May 2017, doi: 10.1093/gigascience/gix019.

[9] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," J. Mach. Learn. Res., vol. 12, pp. 2825–2830, 2011.

[10] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in Adv. Neural Inf. Process. Syst. 32 (NeurIPS), 2019, pp. 8024–8035.

[11] C. Mou, X. Yan, X. Miao, and L. Zhu, "Accuracy of wearable devices in predicting falls in older adults: A systematic review and meta-analysis," Front. Public Health, vol. 14, Art. no. 1778750, Mar. 2026, doi: 10.3389/fpubh.2026.1778750.

[12] S. Altunkaya, "Leveraging feature selection for enhanced fall risk prediction in elderly using gait analysis," Med. Biol. Eng. Comput., vol. 62, pp. 3887–3897, 2024, doi: 10.1007/s11517-024-03180-2.

[13] I. Bargiotas et al., "Preventing falls: The use of machine learning for the prediction of future falls in individuals without history of fall," J. Neurol., vol. 270, no. 2, pp. 618–631, Feb. 2023, doi: 10.1007/s00415-022-11251-3.